Utilizing Generative AI in API Testing

Here are some prompts you can use to leverage Generative AI for various API testing tasks:

1) Creating Payload

Prompt:

Generate a JSON payload for creating a new user in an API. The payload should contain fields like `id`, `name`, `email`, `age`, and `address` with sample values.

2) Creating POJO (Plain Old Java Object) Class from JSON

Prompt:

Generate a Java POJO class for the following JSON:

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john.doe@example.com",
  "age": 30,
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "zipcode": "10001"
  }
}
```

3) Creating Schema from JSON

Prompt:
Generate a JSON Schema for the following JSON response:

```
{
  "productId": 1,
  "productName": "Laptop",
  "price": 999.99,
  "available": true
}
```

4) Add Assertions for the API Response

Prompt:
Write assertions using RestAssured and TestNG for the following API response:

```
{
  "status": "success",
  "message": "User created successfully",
  "userId": 101
}
```

Validate that:

- The status code is 201.
- The status field is "success".
- The userId is an integer."

5) Convert JSON to CSV

Prompt:
Write a Java utility to convert the following JSON array to a CSV file:

```
[
  { "id": 1, "name": "Alice", "email": "alice@example.com" },
  { "id": 2, "name": "Bob", "email": "bob@example.com" }
]
```

6) Convert CSV to JSON

Prompt:
Write a Java program to read a CSV file containing columns `id`, `name`, and `email`, and convert it into a JSON array.

7) Creating Utility to Read Data from JSON File

Prompt:
Write a Java utility to read data from a JSON file and return it as a Java object using Jackson. Assume the JSON contains a list of users with `id`, `name`, and `email` fields.

8) Creating Utility to Read Data from CSV File

Prompt:
Write a Java utility to read a CSV file containing `id`, `name`, and `email` fields and return a list of Java objects.

9) Creating Utility to Read Data from XML File

Prompt:

Write a Java utility to read an XML file and convert it into a Java object using JAXB. The XML file contains user data with fields `id`, `name`, and `email`.